

Avance de Trabajo Final 2

Gestor de biblioteca

Integrantes:

- Tapia Santa Adriana Lucia
- Salazar Camacho Sandro Piero
- Rojas Gamonal Angel Samir
- Ramos Valdivia Gerardo Adrian

Contenido

01 **Resumen AVTF1**

02 **Arquitectura del sistema**

03 **Desarrollo del proyecto**

04 **Git y GitHub**

05 **Gestión de ramas**

06 **Projects e Issues**

07 **Conflictos**

08 **Control de versiones / tags**

09 **Revisiones, sugerencias y comentarios**

10 **Jira**

11 **Conclusiones**

Realidad problemática

Pese al incremento en el consumo de textos digitales en el Perú, existe una crisis en la retención del conocimiento: la falta de herramientas especializadas para la síntesis y reflexión provoca que el lector olvide por completo lo aprendido (Ahorro de Reaprendizaje del 0%). Se requiere una solución que centralice la organización bibliográfica y el análisis crítico, superando la dispersión de las apps de notas y el enfoque superficial de las redes sociales de libros.



Justificación

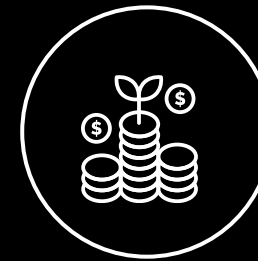
El proyecto se justifica por su capacidad de transformar la lectura de un acto pasivo a un proceso de **alfabetización funcional**, fortaleciendo el pensamiento crítico frente a la inmediatez digital. Basado en principios de psicología cognitiva, la herramienta optimiza la productividad académica y científica al reducir la deuda cognitiva y centralizar el conocimiento disperso. En un entorno dominado por contenidos efímeros, esta solución surge como una urgencia estratégica para preservar el valor de la lectura profunda y garantizar que el aprendizaje sea un activo duradero y no un consumo superficial.

Objetivos

Objetivo general

Diseñar e implementar un software de gestión de libros favoritos que integre una API externa para organizar lecturas, almacenar resúmenes y opiniones personales, con el fin de fortalecer la memoria crítica y la gestión del conocimiento en la era digital.

Objetivos específicos



Analizar las necesidades de los lectores en relación con la gestión del conocimiento personal, elaborando un informe de **requerimientos** funcionales y no funcionales.



Diseñar la **arquitectura** del sistema y la base de datos relacional que soporte usuarios, libros, listas y reseñas, documentando un esquema técnico documentado.



Implementar la integración con una **API** externa de libros para permitir la búsqueda y registro de títulos, desarrollando un módulo funcional de conexión.



Desarrollar un **prototipo interactivo** (MVP) que permita a los usuarios crear listas de favoritos, añadir reseñas y almacenar resúmenes, entregando una versión inicial utilizable dentro del curso.

Marco teórico

01

Estado del Arte: La digitalización ha facilitado el acceso a textos, pero también ha fragmentado la atención. Se prioriza la rapidez sobre el análisis, generando la necesidad de herramientas que fomenten una lectura más profunda y organizada.

02

Memoria y Aprendizaje: Según la psicología cognitiva, la lectura pasiva no basta; la retención requiere procesos activos de organización y asociación. Sin herramientas de apoyo, la sobreinformación actual acelera el olvido del conocimiento adquirido.

03

Comprensión Digital: Leer en pantallas suele ser un proceso no lineal que genera distracción. Para mejorar la comprensión, es vital aplicar estrategias metacognitivas (planificar y monitorear la lectura) que el sistema propuesto busca automatizar.

04

Gestión del Conocimiento: El pensamiento crítico se fortalece al registrar resúmenes y opiniones personales. El sistema centraliza esta información, transformando el consumo efímero de datos en un activo intelectual duradero y estructurado.



Metodología

El proyecto consiste en una investigación aplicada con enfoque cualitativo, orientada a desarrollar una solución tecnológica que resuelva la deficiencia en la retención de información digital. Se fundamenta en el análisis del comportamiento del usuario y la revisión documental para diseñar una herramienta que gestione eficientemente el conocimiento personal.



01

Técnicas e Instrumentos: Se emplea la observación y la revisión académica para identificar cuellos de botella en la toma de notas. El stack tecnológico incluye Angular para el frontend, Spring Boot para el backend y PostgreSQL, integrando además una API externa para la recuperación de datos bibliográficos.

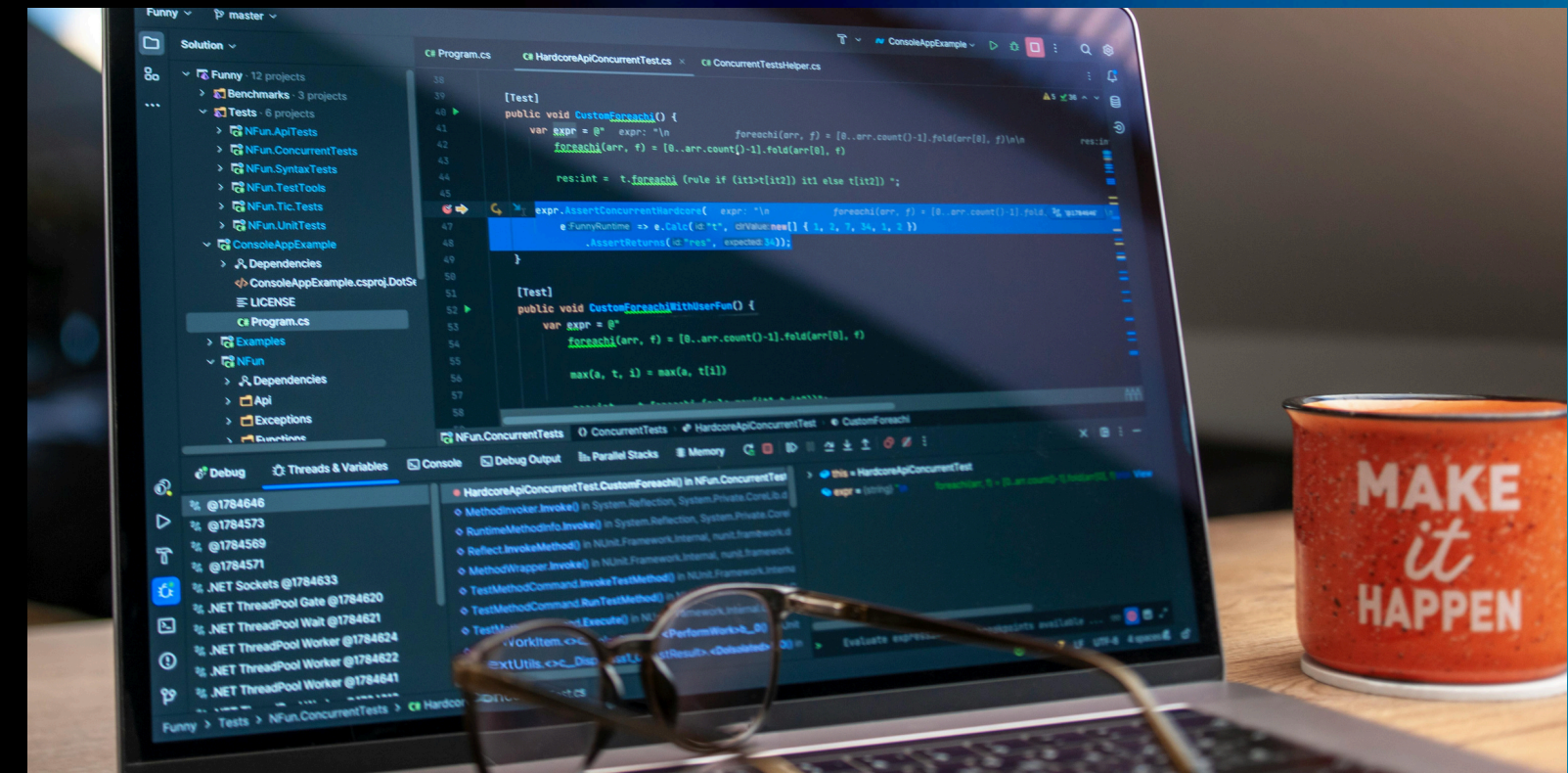
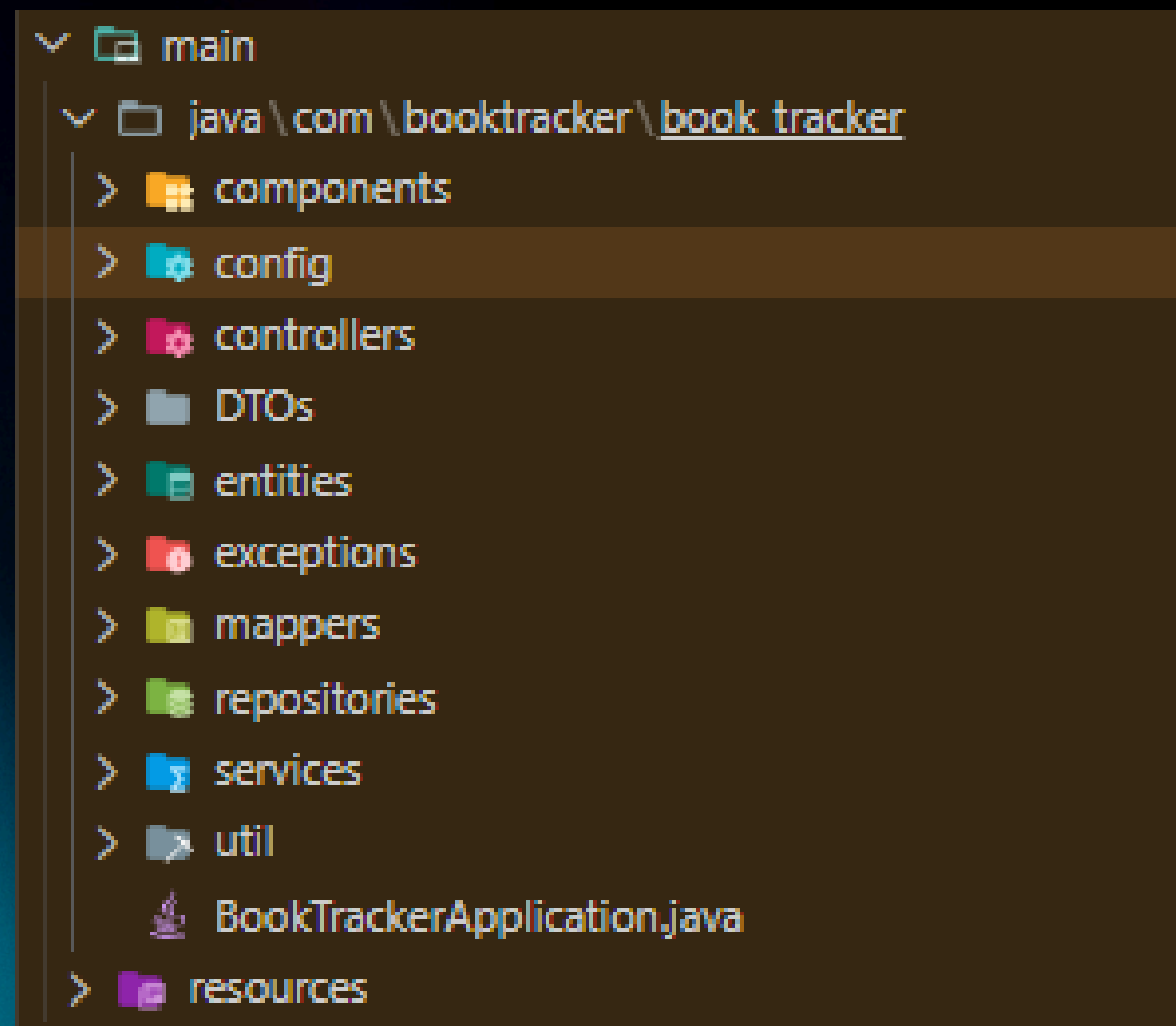
02

Proceso e Ingeniería: Se utiliza un enfoque iterativo para construir un Producto Mínimo Viable (MVP), abarcando fases de análisis, arquitectura cliente-servidor y pruebas. La validación final asegura que el usuario pueda registrar libros, almacenar resúmenes y gestionar sus opiniones de manera funcional y estructurada.

Arquitectura del sistema

Arquitectura en capas

La aplicación utiliza una arquitectura en capas para separar responsabilidades y facilitar el mantenimiento del sistema.



Componentes

- Controllers
- Services
- Repositories
- Entities
- DTOs

Estrategia de Persistencia y Base de Datos

Uso de PostgreSQL con Spring Data JPA e Hibernate para una gestión relacional eficiente.

Desarrollo de trabajo

Flujo de Trabajo (Git/GitHub): Control de versiones profesional mediante el uso de ramas (branches) y commits descriptivos, facilitando la integración de nuevas funcionalidades construidas.



Módulos implementados

Módulo de autenticación
Módulo de libro
Módulo de listas
Módulo de reseñas



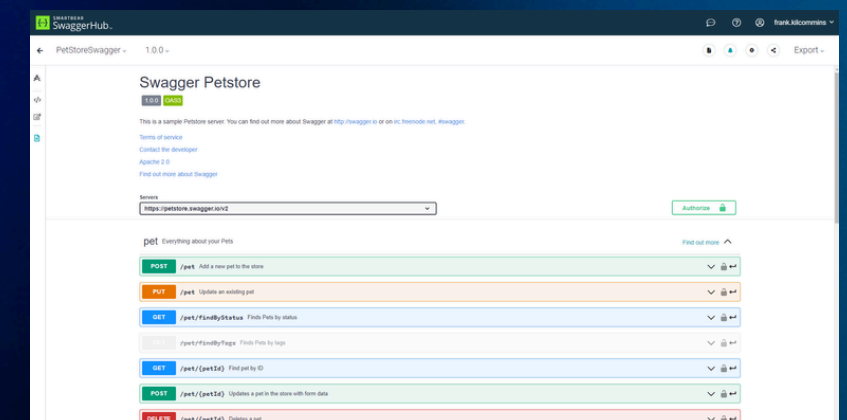
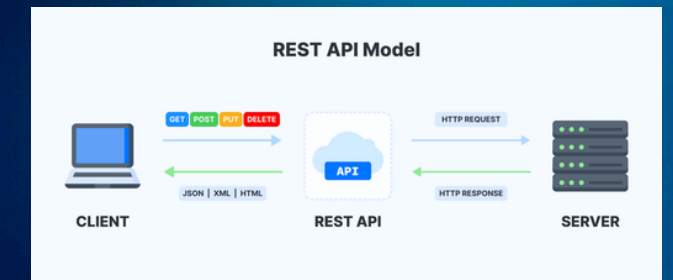
API Rest

Endpoints implementados. Los endpoints REST permiten realizar operaciones CRUD utilizando respuestas JSON.



Swagger/OpenAPI

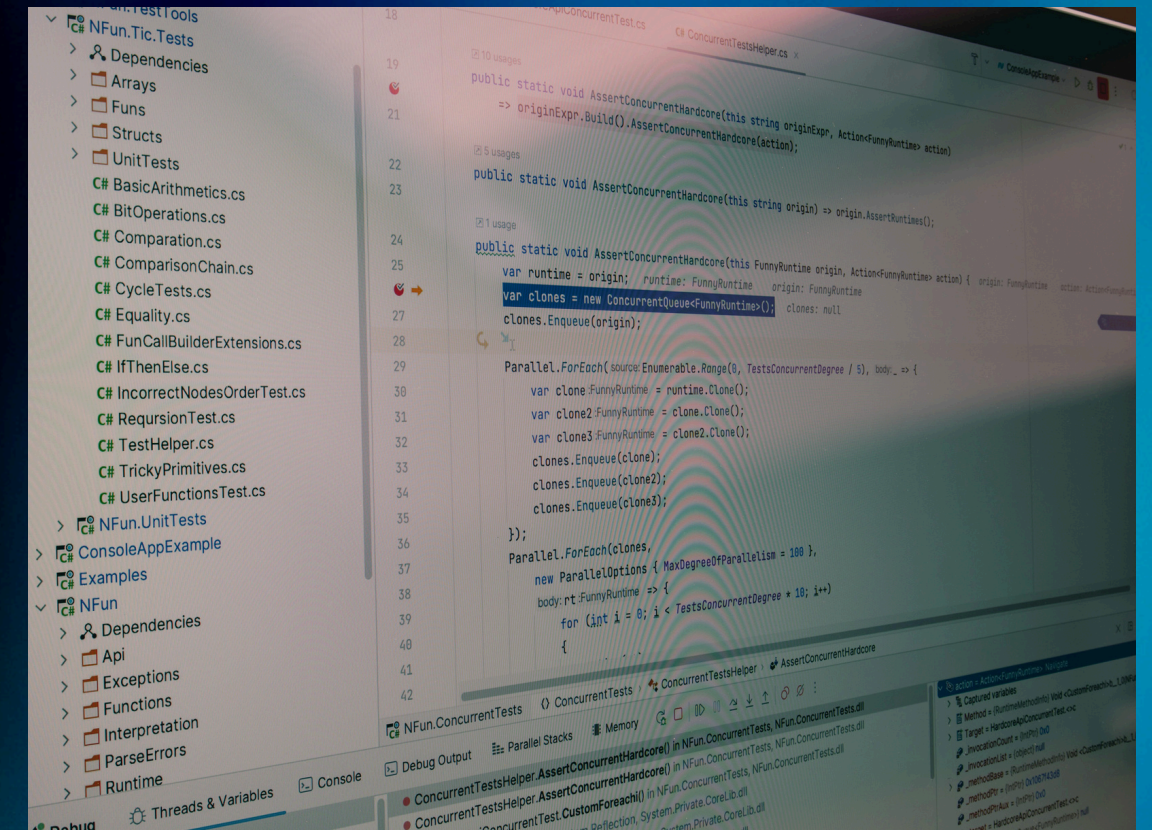
Se integró Swagger/OpenAPI para documentar automáticamente todos los endpoints REST.



Herramienta Git y Github

Git: Es el motor (local). Permite capturar "fotografías" (commits) del código para volver atrás si algo falla y trabajar en ramas (branches) sin alterar la versión principal.

GitHub: Es la plataforma (nube). Facilita el trabajo en equipo, aloja el repositorio remoto y permite la integración mediante Pull Requests.



01

Branching

Se crea una rama específica para una nueva funcionalidad (ej. feature/api-books) para mantener limpio el código estable (main)

02

Staging & Commit

Se seleccionan los archivos modificados (git add) y se guardan con un mensaje descriptivo (git commit -m "Integración de API externa").

03

Push & PR

Se sube la rama a GitHub (git push) y se abre un Pull Request (PR) para que el equipo revise el código antes de unirlo al proyecto

04

Merge

Una vez aprobado, el código se fusiona con la rama principal, asegurando que cada cambio esté validado.



Gestión de ramas

BRANCH / TAG	GRAPH	COMMIT MESSAGE	AUTHOR	CHANGES	COMMIT DATE / T...	SHA
docs/conflicto-...		docs: documentar otra resolución de conflicto local	You	1	anteayer	677c768
feature/local-a		fix: resolver conflicto local	You	0	anteayer	027526d
feature/local-b		test: cambio local B	You	1	anteayer	78638a3
		test: cambio local A	You	1	hace 3 días	8a639fb
		docs: documentar resolución de conflicto	You	1	hace 3 días	6f45967
feature...		fix: resolución de excepción ideal	You	1	hace 3 días	4172e3e
		fix: resolver conflicto con develop	You	0	hace 3 días	a3f6ee3
develop		Merge pull request #41 from adrianawzw/feature/te...	Gerardo Adrian Ram...	0	hace 3 días	d436508
feature/test...		test: provocar conflicto en UserServiceImpl	You	1	hace 4 días	9e6e8f4
feature/migraci...		fix: configuracion basica de postgresql	You	2	hace 4 días	9d2840b
		Implementacion de Excepciones y Token de segurid...	Gerardo Ramos	33	hace 6 días	86bb6bc
		Merge pull request #38 from adrianawzw/feature/se...	spsalazarc-alt	0	la semana pasada	53f489f
feature/services...		feat: services, controllers y OpenLibrary	Sandro	3	la semana pasada	79f11b0
feature/service...		feat: services	Sandro	24	la semana pasada	742e79c
		Merge pull request #28 from adrianawzw/feature/ac...	Angelrg06	0	hace 2 semanas	c9f82cc
		fix: agregar import faltante y corregir error	Angelrg06	1	hace 2 semanas	4b82f36

01

Ramas utilizadas

- main
- develop
- feature/servicesAll
- Exception_JWT

02

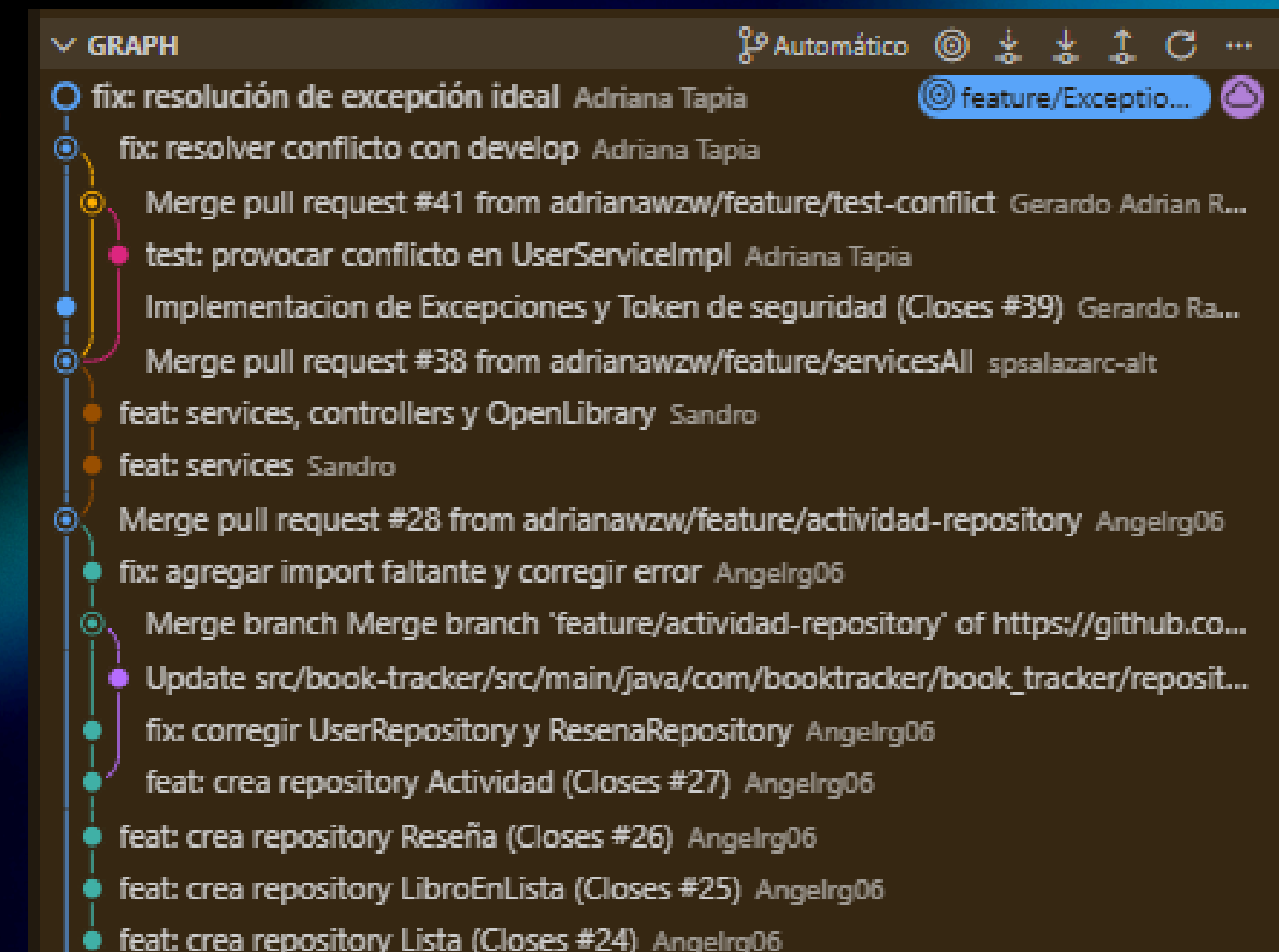
Desarrollo paralelo

Cada integrante trabajó en ramas feature/ independientes para evitar afectar la rama principal.

03

Pull Requests (PR)

Los cambios fueron integrados mediante Pull Requests revisados por el equipo con comentarios descriptivos.



GitHub Projects e Issues

Gestión ágil: Se utilizó GitHub Projects para organizar tareas mediante un tablero Kanban.

Tablero: Es la herramienta visual para gestionar el flujo de trabajo. En GitHub se encuentra en la pestaña "Projects".

01

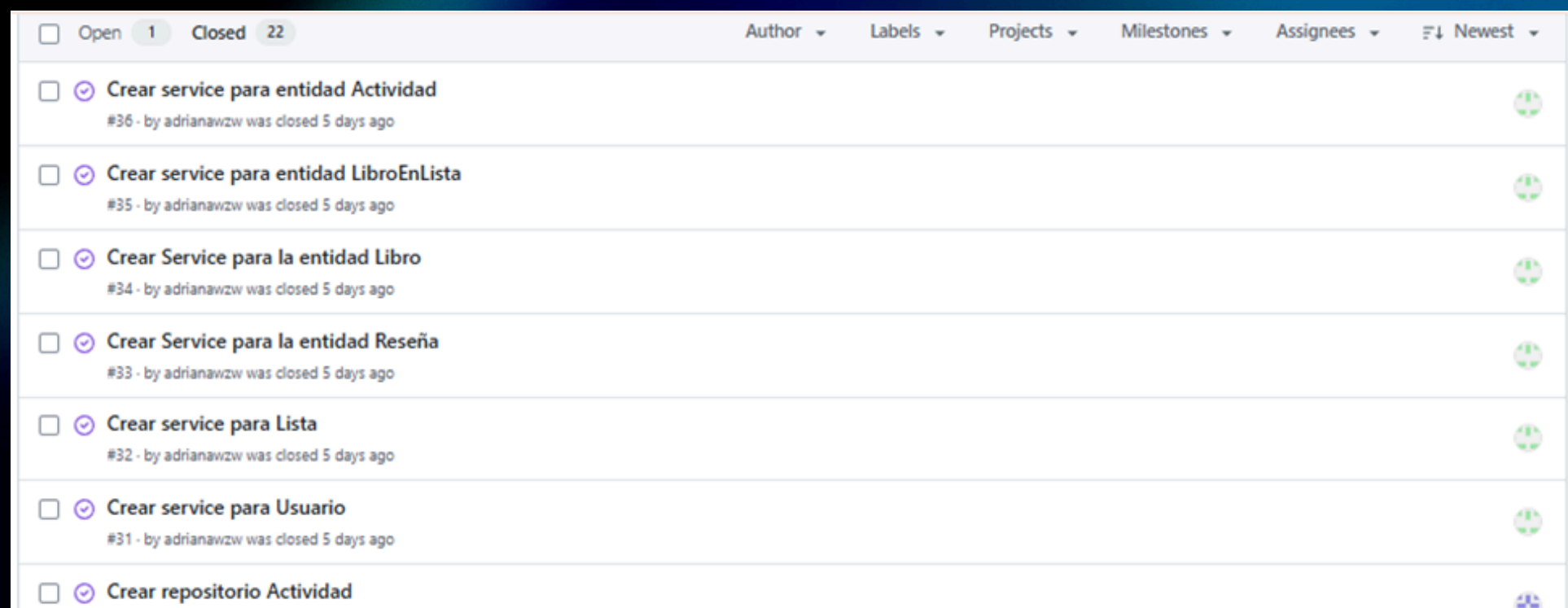
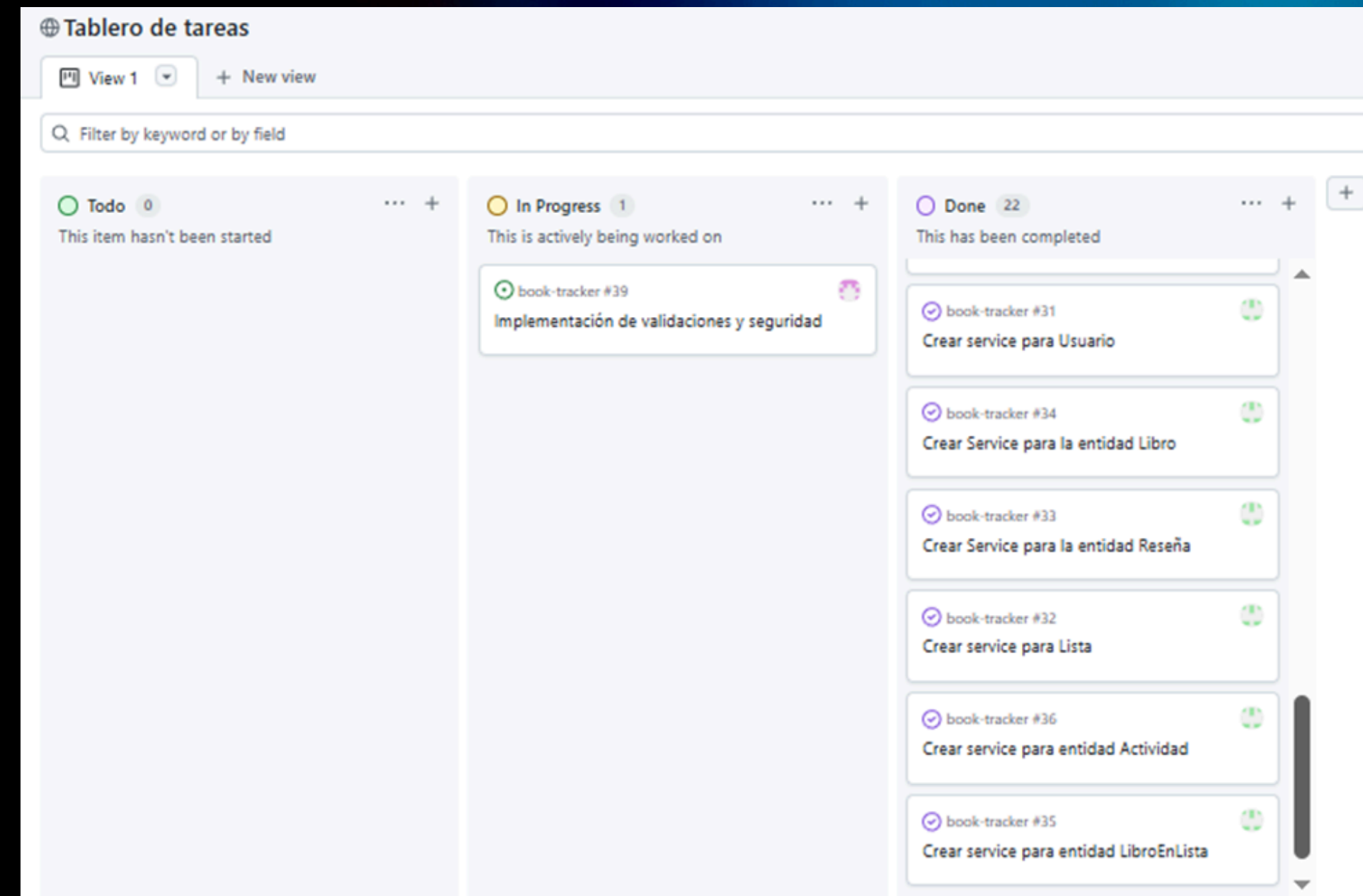
Columnas usadas

- To Do
- In Progress
- Done

02

Issues

Son las unidades de trabajo del proyecto. Las tareas fueron registradas mediante GitHub Issues y vinculadas a Pull Requests.



Gestión de conflictos

Conflicto provocado: Se generó un conflicto modificando simultáneamente un mismo archivo desde diferentes ramas.

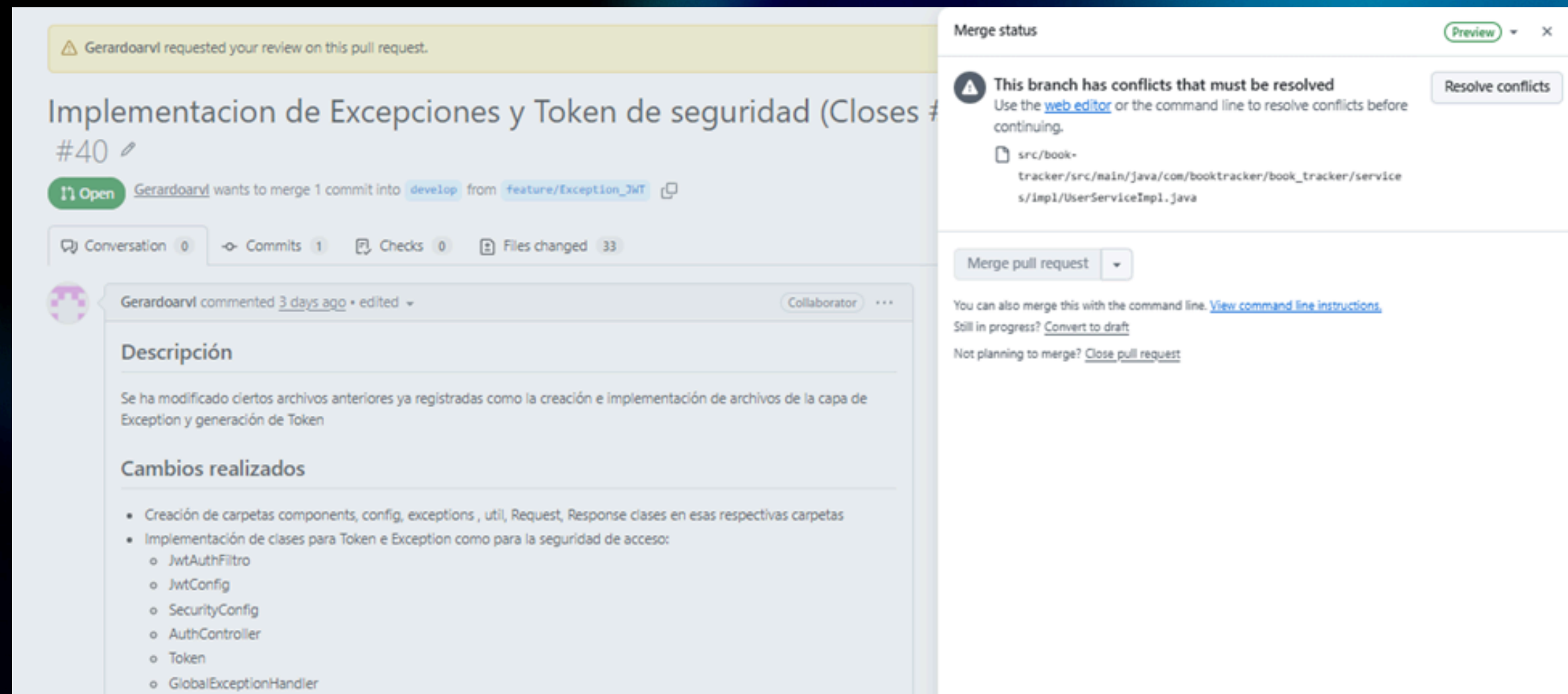
Resolución: El conflicto fue resuelto manualmente utilizando Visual Studio Code.

Comandos utilizados:

- git merge

Ramas involucradas:

- develop
- feature/Exception_JWT
- feature/test-conflict



Gestión de conflictos

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (develop)
$ git checkout feature/Exception_JWT
Switched to branch 'feature/Exception_JWT'
Your branch is up to date with 'origin/feature/Exception_JWT'.
```

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT)
$ git merge develop
Auto-merging src/book-tracker/src/main/java/com/booktracker/book_tracker/services/impl/UserServiceImpl.java
CONFLICT (content): Merge conflict in src/book-tracker/src/main/java/com/booktracker/book_tracker/services/impl/UserServiceImpl.java
Automatic merge failed; fix conflicts and then commit the result.
```

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT|MERGING)
$ |
```

```
36 |         .map(UserMapper::toDTO)
37 |         .toList();
38 |     }
39 |
40 | @Override Sandro, la semana pasada • feat: services ...
41 | public UserResponseDTO obtenerPorId(Long id) {
42 |
43 |
44 |     User usuario = userRepository.findById(id)
45 | <<<<<<< HEAD (Cambio actual)
46 |         .orElseThrow(() -> new ResourceNotFoundException("Usuario no encontrado"));
47 | =====
48 |
49 |         .orElseThrow(() -> new IllegalStateException("Usuario inválido"));
50 | >>>>>>> develop (Cambio entrante)
51 |
52 |     return UserMapper.toDTO(usuario);
53 | }
54 |
```

 Gerardoarvl requested your review on this pull request. [Add your review](#)

Implementacion de Excepciones y Token de seguridad (Closes #39)

Awaiting approval Code

#40 

 Open Gerardoarvl wants to merge 2 commits into develop from feature/Exception_JWT 

Conversation 0

Commits 2

Checks 0

Files changed 33

+891 -74 

 Gerardoarvl commented 3 days ago • edited

Collaborator ...

Descripción

Reviewers

 Angelrg06

 adrianawzw

 spsalazarc-alt

Control de versiones y tags

Versionamiento:

Se utilizaron tags para identificar versiones importantes del proyecto.

Comando

git tag -a v1.0 -m "Primera versión"

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDe
ker (develop)
$ git tag -a v1.0 -m "Versión final de AVTF2 del proyecto"

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDe
ker (develop)
$ git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 191 bytes | 191.00 KiB/s, done.
```

Releases / v1.0

Versión 1.0 - Entrega Final AVTF2

Latest

Compare



adrianawzw released this yesterday · 22 commits to main since this release v1.0 799c2d5

Primera versión del sistema Book Tracker.
Incluye autenticación, gestión de libros y reseñas.

Assets 2

Source code (zip)	yesterday
Source code (tar.gz)	yesterday

Commits

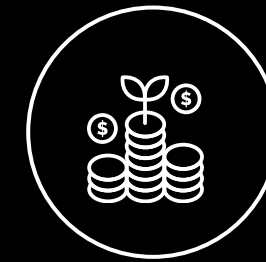
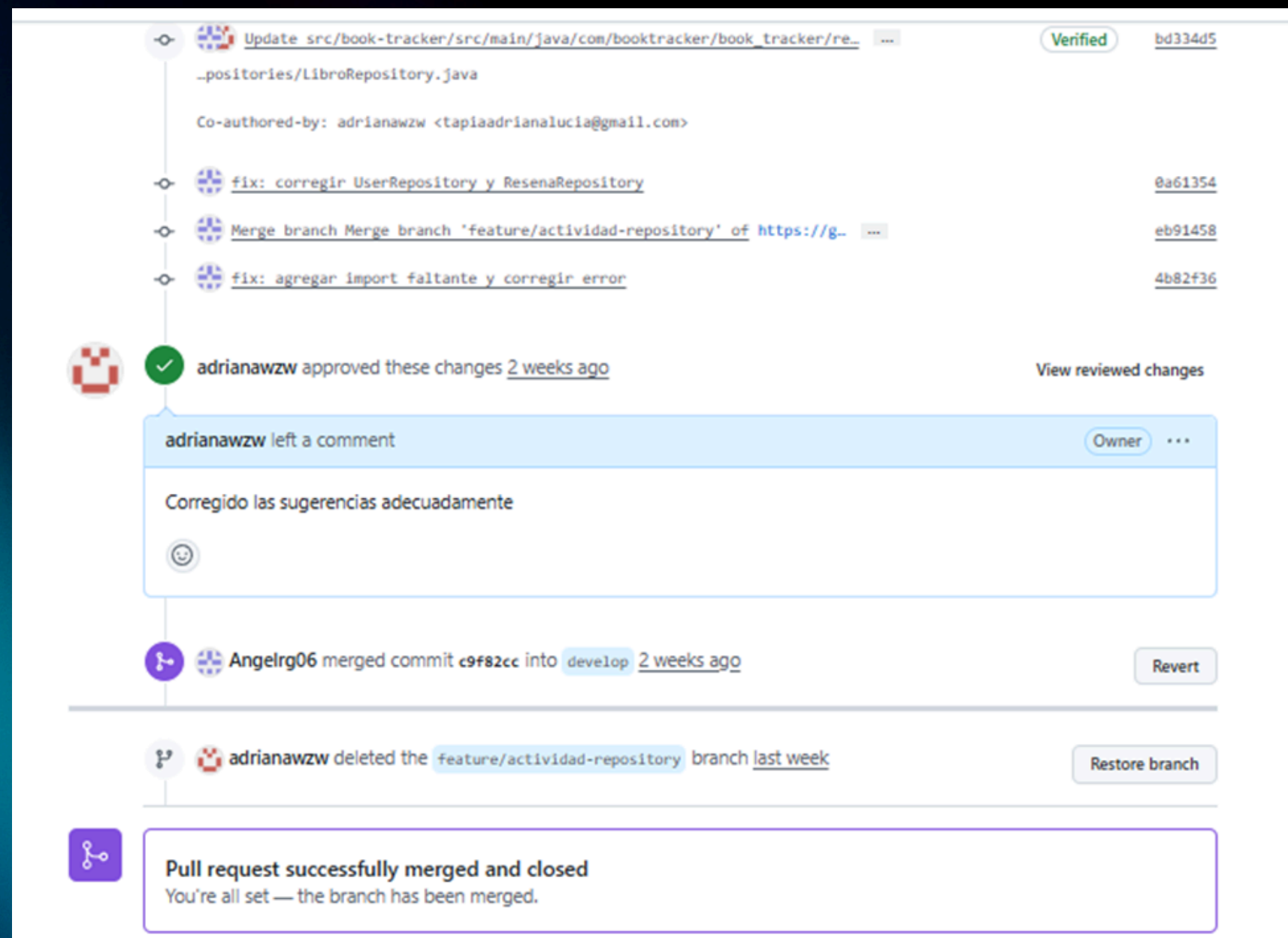
Durante el avance 2 se realizaron más de 35 commits descriptivos.

The screenshot displays the GitHub commit history for the repository. It is organized into sections by date:

- Commits on May 25, 2026:** A single commit titled "Implementacion de Excepciones y Token de seguridad (Closes #39)" by Gerardoarvl, committed last week. The commit hash is 86bb6bc.
- Commits on May 28, 2026:** Three commits are shown:
 - "test: provocar conflicto en UserServiceImpl" by adrianawzw, committed 5 days ago. Hash: 9e6e8f4.
 - "Merge pull request #41 from adrianawzw/feature/test-conflict" by Gerardoarvl, authored 4 days ago. Hash: d436508. This commit is marked as "Verified".
 - "fix: resolver conflicto con develop" by adrianawzw, committed 4 days ago. Hash: a3f6ee3.
- Commits on May 29, 2026:** One commit is shown:
 - "fix: resolución de excepción ideal" by adrianawzw, committed 4 days ago. Hash: 4172e3e.
- Commits on Jun 1, 2026:** One commit is shown:
 - "Merge pull request #40 from adrianawzw/feature/Exception JWT" by adrianawzw, authored 4 minutes ago. Hash: 799c2d5. This commit is marked as "Verified".

Revisiones, sugerencias y comentarios

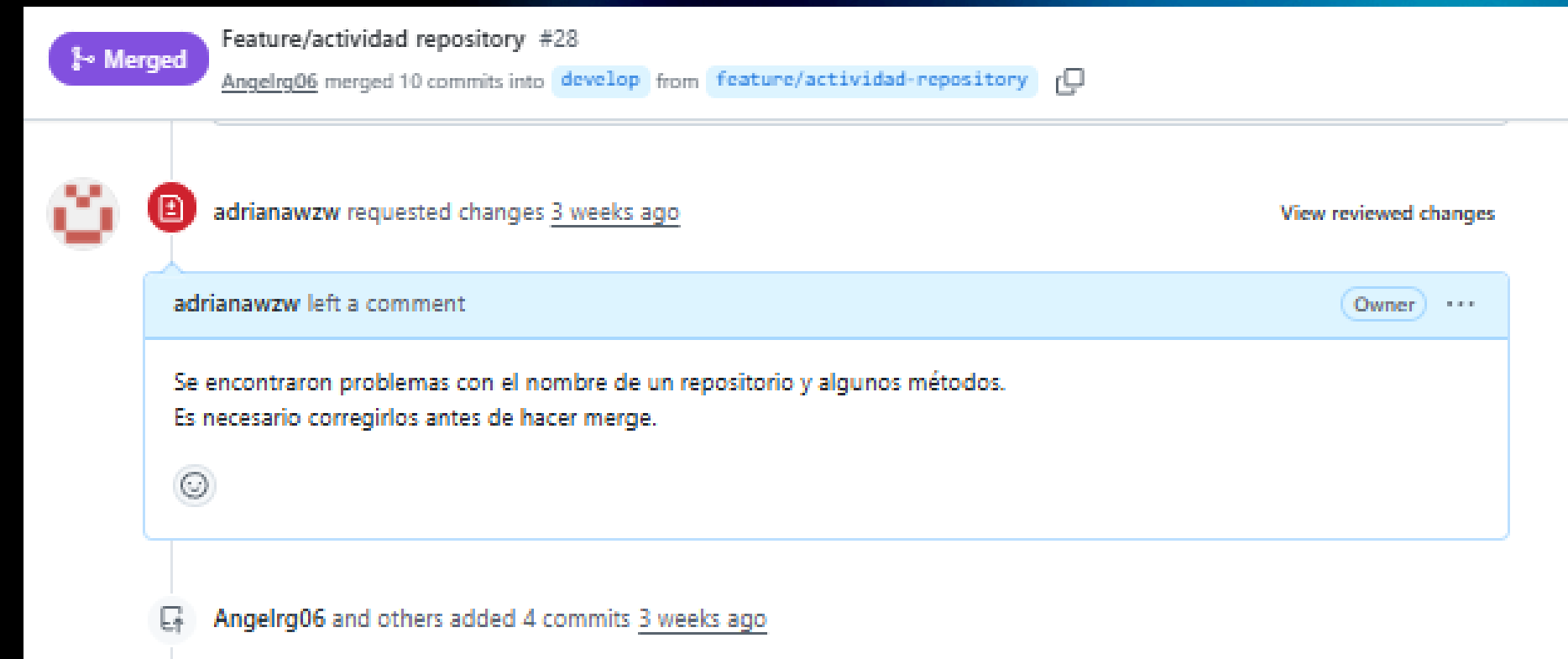
En GitHub, las revisiones de Pull Request (PR) sirven para que otros desarrolladores revisen tu código antes de unirlo (merge) a otra rama, normalmente develop o main.



Revisiones

Cuando alguien revisa el código del PR puede:

- aprobarlo
- pedir cambios
- dejar comentarios



Sugerencias

GitHub permite dejar cambios sugeridos directamente.

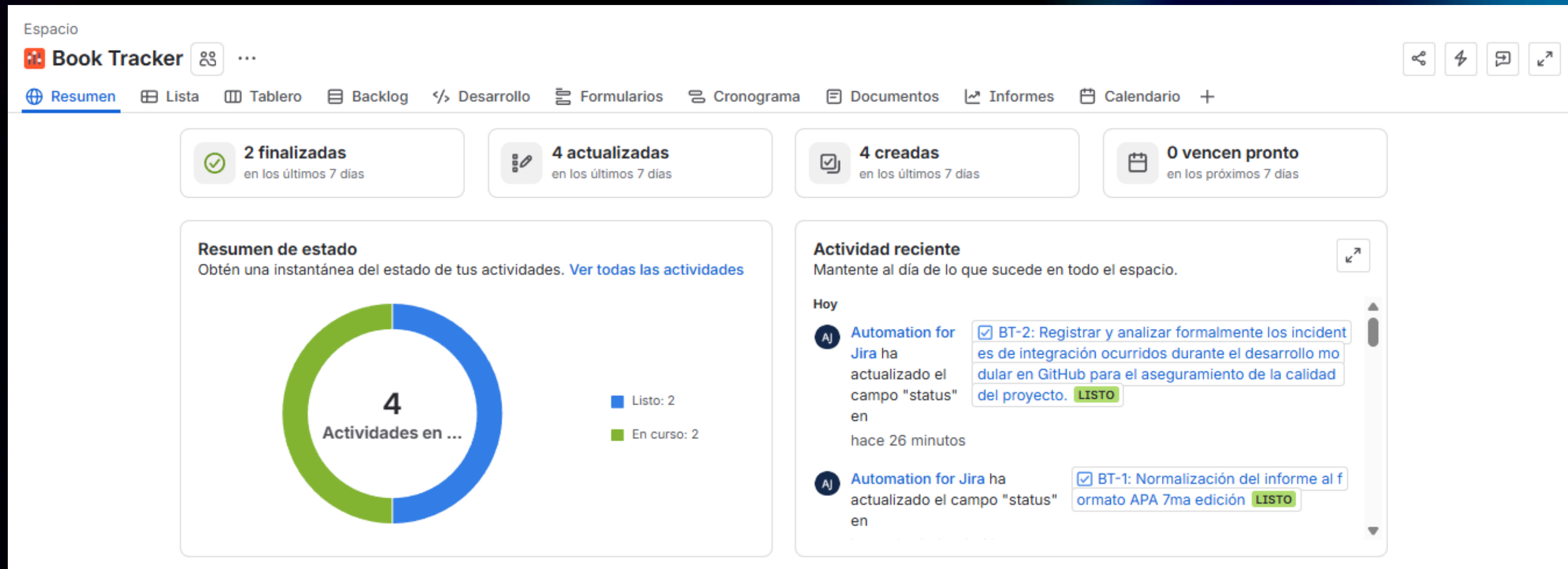


Comentarios

Los comentarios son mensajes que se dejan sobre líneas específicas del código.

Gestión de proyectos con JIRA

Se utilizaron herramientas diferenciadas para la gestión del proyecto y el desarrollo técnico. Jira fue empleado para la planificación general, asignación de responsabilidades y seguimiento de actividades globales, mientras que GitHub Projects e Issues se utilizaron para tareas específicas de programación e integración de código.



LISTA DE ACTIVIDADES

La lista de actividades muestra todas las tareas creadas en el proyecto Book Tracker, identificadas con las claves BT-1, BT-2 y BT-3. Cada tarea cuenta con un responsable asignado y un estado actual que indica su progreso dentro del flujo de trabajo. Esta lista permite al equipo tener una visión clara de qué actividades están pendientes, en desarrollo o ya completadas.

Book Tracker

Preguntar a Atlassian Intelligence

Buscar actividad

SR

AT

GV

S

Filtro

Agrupar

Filtros guardados

	Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución	
	BT-1 Normalización del informe al formato APA 7ma edición	AT Adriana Tapia	SR Samir Rojas	Medium	LISTO	Listo	31
	BT-2 Registrar y analizar formalmente los incidentes de integración ocurridos dura...	SR Samir Rojas	SR Samir Rojas	Medium	LISTO	Listo	31
	BT-3 Especificación de requisitos funcionales y no funcionales del backend	GV Gerardo Adrian Ramo...	SR Samir Rojas	Medium	EN CURSO	Sin resolver	31
	BT-4 Elaboración del diccionario de datos y reglas de integridad de la BD	S Sandro.dll	SR Samir Rojas	Medium	EN CURSO	Sin resolver	31

+ Crear

4 de 4

BACKLOG

El backlog es el espacio donde se almacenan las tareas que aún no han sido planificadas o asignadas a un sprint. Aquí se pueden priorizar las actividades según su importancia y urgencia. Desde el backlog, el equipo decide qué tareas se moverán al sprint actual para su desarrollo, permitiendo una gestión ágil y ordenada del proyecto.

Book Tracker

...

Q Buscar en el ba...

SR

AT

GV

S

Filtro

...

☐ Sprint1 - Avance de Proyecto 2 4 may – 3 jun (4 actividades)

000Completar sprint...

Completar la documentación técnica del Avance 1, incluyendo la normalización APA, el reporte de conflictos, la especificación de requisitos y el diccionario de datos de la base de datos local.

☒ BT-1	Normalización del informe al formato APA 7ma edición	LISTO	-	=	AT
☒ BT-2	Registrar y analizar formalmente los incidentes de integración ocurridos durante el desarrollo modular en GitHub para el aseguram...	LISTO	-	=	SR
☒ BT-3	Especificación de requisitos funcionales y no funcionales del backend	EN CURSO	-	=	GV
☒ BT-4	Elaboración del diccionario de datos y reglas de integridad de la BD	EN CURSO	-	=	S

+ Crear

☐ Backlog (0 actividades)

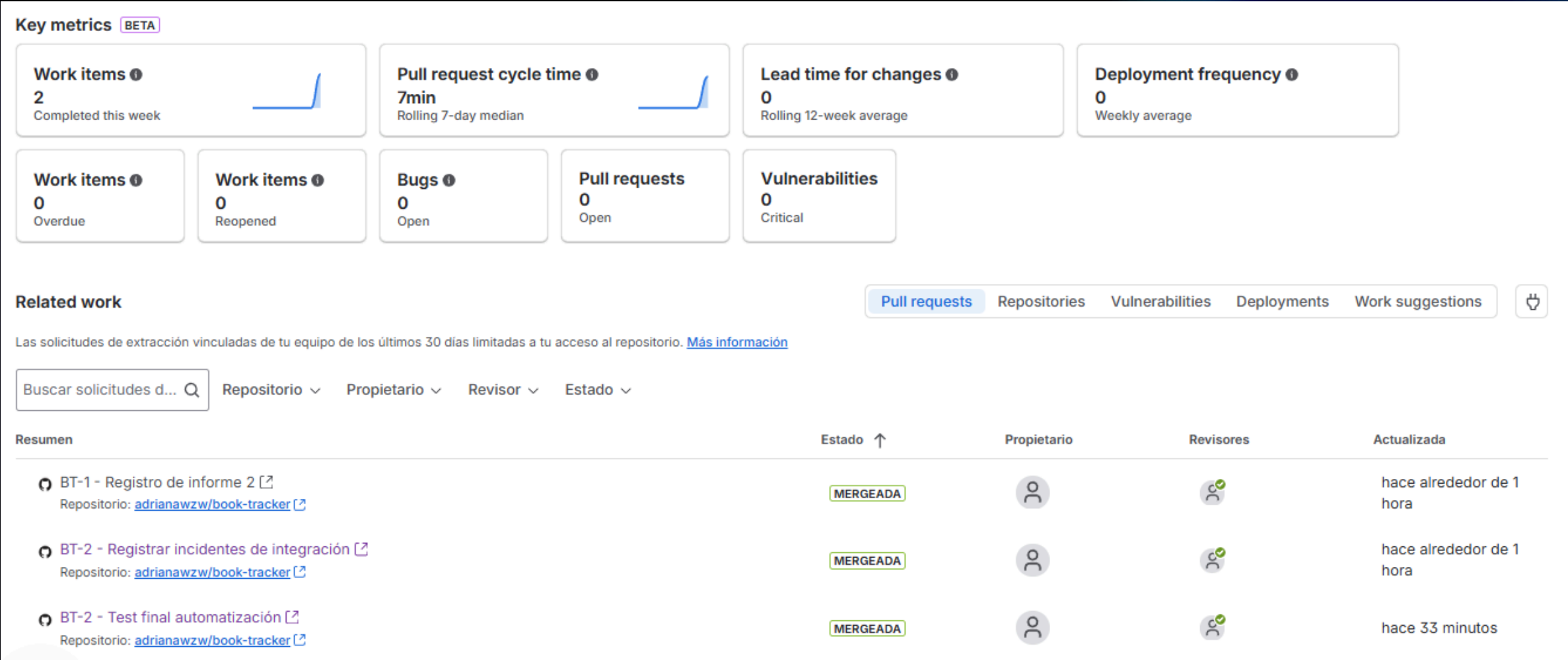
000Crear sprint

Tu backlog está vacío.

+ Crear

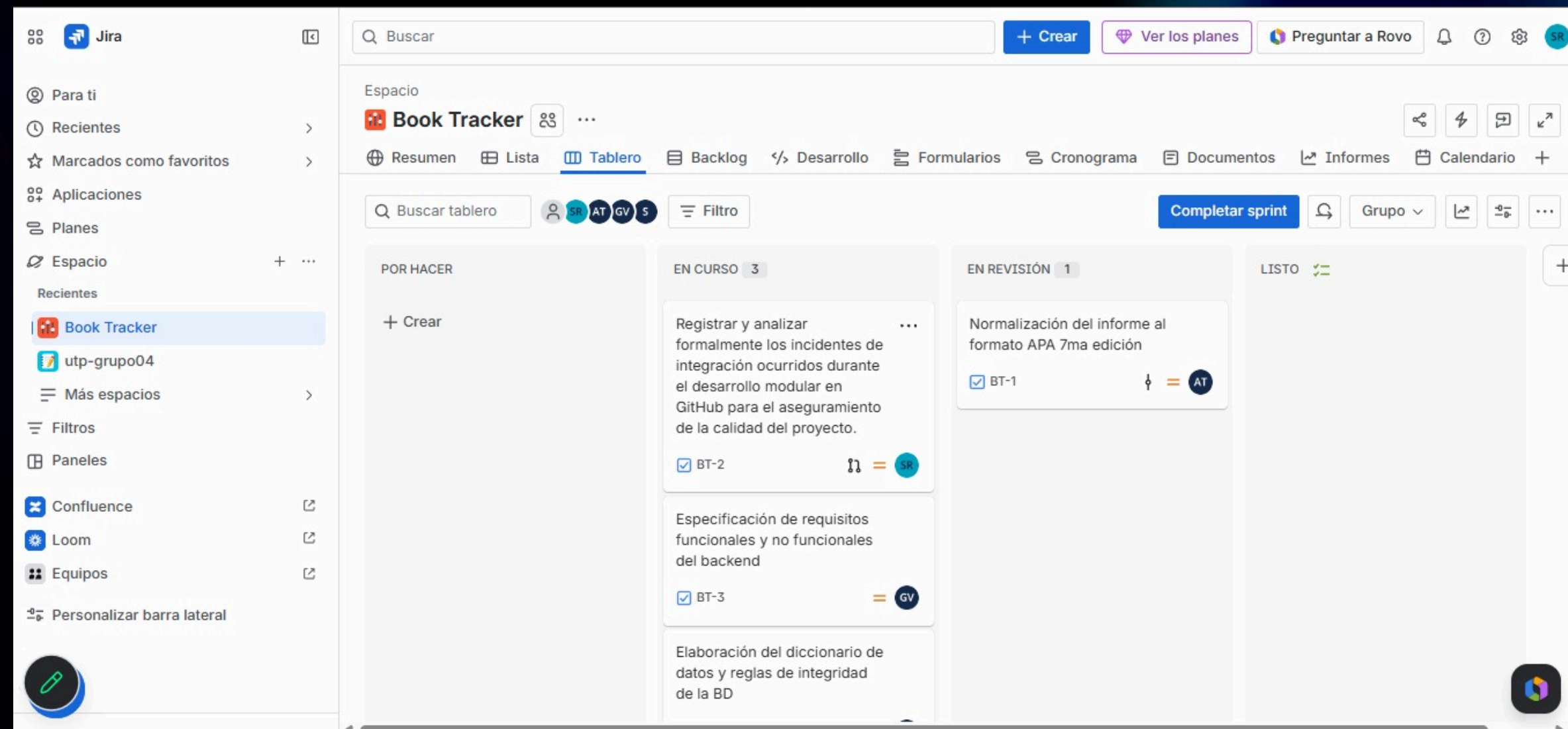
DESARROLLO

En la sección de Desarrollo se muestra la integración entre Jira y GitHub. Aquí aparecen vinculadas las ramas, commits y pull requests relacionados con cada tarea. Por ejemplo, para la tarea BT-2 se puede visualizar la rama feature/incidentes, los commits realizados y el pull request #42. Esta integración permite mantener la trazabilidad entre el código fuente y las tareas planificadas.

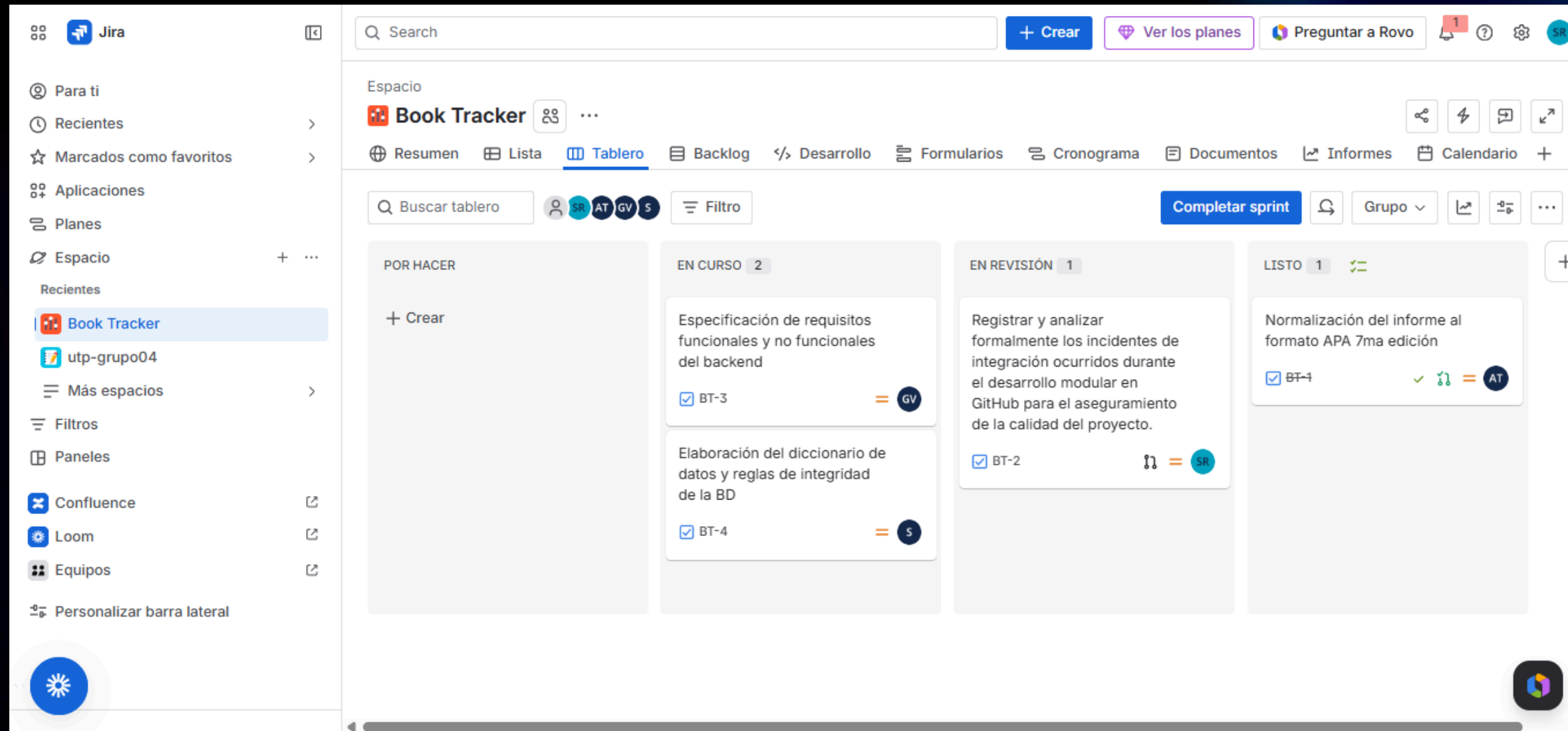


Automatización del tablero

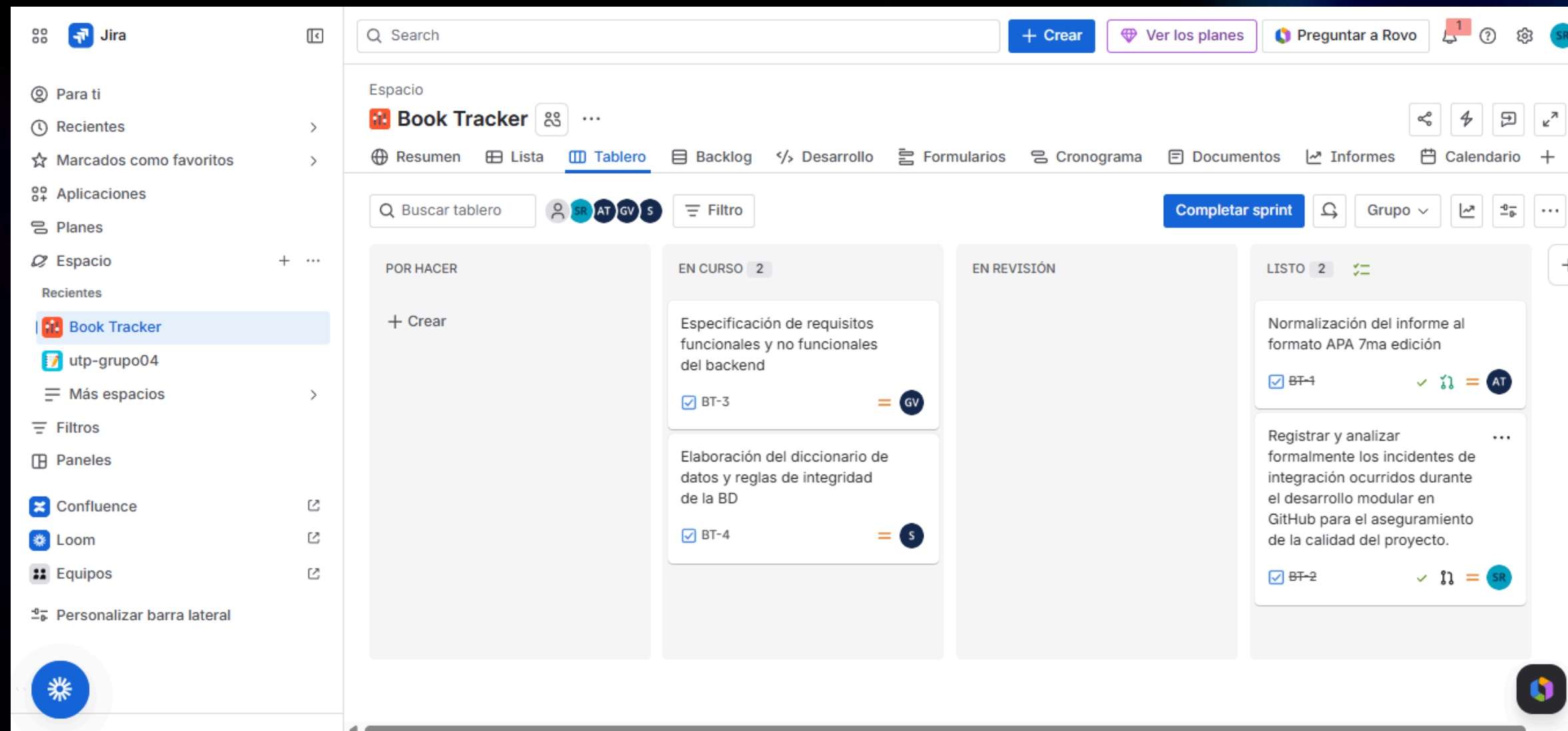
El tablero de Jira muestra el flujo de trabajo organizado en columnas que representan los estados: Por hacer, En curso, En revisión y Listo. Las tareas cambian de estado automáticamente gracias a tres reglas de automatización configuradas. La primera regla: cuando se crea una rama en GitHub, la tarea pasa a "En curso". La segunda regla: cuando se abre un Pull Request, la tarea pasa a "En revisión". La tercera regla: cuando se fusiona (merge) el Pull Request, la tarea pasa a "Listo". Todo este proceso es automático y no requiere intervención manual.



AUTOMATIZACIÓN DEL TABLERO



AUTOMATIZACIÓN DEL TABLERO



Conclusiones

- Se implementó correctamente el trabajo colaborativo mediante Git y GitHub.
- Los módulos principales del sistema fueron desarrollados exitosamente.
- Swagger/OpenAPI mejoró la documentación y pruebas de la API.
- GitHub Projects facilitó la organización del desarrollo del código del equipo.
- Jira permitió la organización y monitoreo de tareas del proyecto.
- La resolución de conflictos permitió fortalecer el flujo de integración.